Unity Manual

2D game creation workflow

Before you create a 2D game, you need to decide on a game perspective and an art style.

To create a 2D game, set up your Unity project and then familiarize yourself with the relevant concepts in the following order:

1. Fundamentals
2. Scripting
3. SpritesA 2D graphic objects. If you are used to working in 3D, Sprites are essentially just standard textures but there are special techniques for combining and managing sprite textures for efficiency and convenience during development. More info
   See in Glossary
4. Building in-game environments
5. Character animation
6. Graphics
7. Physics 2D
8. Audio
9. User interface
10. Profiling, optimizing and testing

Reject All Accept All Cookies

By clicking “Accept Cookies”, you agree to the storing of cookies on your device to enhance site navigation, analyze site usage, and assist in our marketing efforts.

×

See in Glossary are fundamental objects in Unity that represent characters, props, scenery, and more. Every object in your game is a GameObject.

GameObjects represent the items in your game; the space in which you place them to build your level is called a sceneA Scene contains the environments and menus of your game. Think of each unique Scene file as a unique level. In each Scene, you place your environments, obstacles, and decorations, essentially designing and building your game in pieces. More info
See in Glossary. Scenes in Unity are always 3D; when you make a 2D game in Unity, you typically choose to ignore the third dimension (the z-axis) but you can also use it in special cases, for example when making 2.5D games.

The behavior of GameObjects is defined by blocks of functionality called components. The following components are fundamental for 2D games:

- Transform: the **Transform component**A Transform component determines the Position, Rotation, and Scale of each object in the scene. Every GameObject has a Transform. More info
  See in Glossary determines the Position, Rotation, and Scale of each GameObject in the scene. Every GameObject has a Transform component.
- Sprite RendererA component that lets you display images as Sprites for use in both 2D and 3D scenes. More info
  See in Glossary: the Sprite Renderer component renders the Sprite and controls how it looks in a scene.
- CamerasA component which creates an image of a particular viewpoint in your scene. The output is either drawn to the screen or captured as a texture. More info
  See in Glossary: devices that capture and display the world to the player. Marking a Camera as **Orthographic** removes all perspective from the Camera's view. This is mostly useful for making isometric or 2D games.
- Collider 2D: this component defines the shape of a 2D GameObject for the purposes of physical **collisions**A collision occurs when the physics engine detects that the colliders of two GameObjects make contact or overlap, when at least one has a Rigidbody component and is in motion. More info
  See in Glossary. See 2D Physics.

Components are UI representations of C# classes; you can use scripts to change and interact with components, or create new ones. See the Scripting section for more details.

Scripting

All 2D games need scriptsA piece of code that allows you to create your own Components, trigger game events, modify Component properties over time and respond to user input in any way you like. More info
See in Glossary. Scripts respond to input from the player and arrange for events in the gameplay to happen when they should.

For details on how to use scripts in Unity see Scripting Overview. Also see the Unity Learn Beginner Scripting course.

Scripts are attached to GameObjects, and any script you create inherits from the MonoBehaviour class.

Sprites

Sprites are 2D graphic objects. You use Sprites for all types of 2D games. For example, you can import an image of your main character as a Sprite.

A character Sprite
A character Sprite

You can also use a collection of Sprites to build a character. This allows you greater control over the movement and animation of your characters.

Multiple Sprites that make up the parts of a character, displayed in the Sprite Editor

By clicking "Accept Cookies", you agree to the storing of cookies on your device
to enhance site navigation, analyze site usage, and assist in our marketing efforts.

Use the Sprite Renderer component to render your Sprites. For example, you can use the Sprite Renderer to change the color and opacity of a Sprite.

Adjusting the color of a Sprite with the Sprite Renderer
Adjusting the color of a Sprite with the Sprite Renderer

See the Introduction to the Sprite Renderer Learn tutorial. Sorting SpritesBy organizing Sprites in layers, you can create an illusion of depth. You can sort Sprites according to many strategies. See Sorting Sprites for full details. For example, you might sort Sprites along the y-axis, so that Sprites that are higher up are sorted behind Sprites that are lower, to make the Sprites that are higher appear further away than the Sprites that are lower.

Sprites sorted along the y-axis
Sprites sorted along the y-axis

To set the overlay order of Sprites, use Sorting Layers.

To group GameObjects with Sprite Renderers, and control the order in which they render their Sprites, use Sorting Groups.

Sprite Atlas

You can use a Sprite Atlas**Graphics:** A utility that packs several sprite textures tightly together within a single texture known as an atlas. More info. **2D:** A texture that is composed of several smaller textures. Also referred to as a texture atlas, image sprite, sprite sheet or packed texture. More info.
See in Glossary to consolidate several Textures into a single combined Texture. This optimizes your game and saves memory. For example, you can add all your Sprites associated with a particular character or purpose to a Sprite Atlas.

A Sprite Atlas
A Sprite Atlas

See the Introduction to the Sprite Atlas Learn tutorial.

Building in-game environments

Environment design refers to the process of building your game's levels and environments. You can combine the environment design tools in this section in whichever way makes the most sense for your game; for example, you can make a top-down game using only 9-slice, or you can make a side on platformer with Tilemap and SpriteShape.

9-slicing

9-slicing is a 2D technique that allows you to reuse an image at various sizes without needing to prepare multiple assets. Unity can dynamically stretch and tile designated parts of a Sprite to allow one Sprite to serve as the border or background for UI elements of many sizes. See 9-slicing Sprites.

For example, you could use 9-slicing to stretch a Sprite to shape when you build a 2D level.

A 9-sliced Sprite, split into nine sections
A 9-sliced Sprite, split into nine sections

See the Using 9-Slicing for Scalable Sprites Learn tutorial.

Tilemap

The TilemapA GameObject that allows you to quickly create 2D levels using tiles and a grid overlay. More info

By clicking "Accept Cookies", you agree to the storing of cookies on your device
to enhance site navigation, analyze site usage, and assist in our marketing efforts.

See the Introduction to Tilemaps Learn tutorial.

2D Tilemap Extras

To add some extra Tilemap assets to your Project, install the 2D Tilemap Extras package. This package contains reusable 2D and Tilemap **Editor scripts**C# source files composed entirely of code that runs in the Unity Editor only and not in the runtime Player build. Keep such scripts in dedicated Editor assemblies either by placing them in a parent folder called Editor or creating an Editor-only assembly definition.
See in Glossary that you can use for your own Projects. You can customize the behavior of the scripts to create new Brushes that suit different scenarios.

Isometric Tilemaps

For games with isometric perspective, you can create Isometric Tilemaps.

SpriteShape

In a similar way to a vector drawing tool, SpriteShape provides a more flexible way to create larger Sprites, such as organic-looking landscapes and paths. See the Sprite Shape Profile.

A path created in SpriteShape
A path created in SpriteShape

See the Working with SpriteShape tutorial.

Character animation

There are three different ways you can animate 2D characters:

2D animation type	Used for
Frame-by-frame	Artistic reasons, if you want your game to have a classic animation art style. Frame-by-frame animation is relatively resource-intensive, both to make and to run.
Cutout	Smooth skeletal animation, when the characters don't require realistic articulation.
Skeletal	Smooth skeletal animation where Sprites bend according to the bone structure. Use this when the characters need a more organic feel.

Frame-by-frame

Frame-by-frame animation is based on the traditional cel animation technique of drawing each moment of an animation as individual images, which are played in fast sequence, like flipping pages on a flipbook.

For information, refer to Sprite Swap in the 2D Animation package manual.

Frame-by-frame animation in the Sprite Editor
Frame-by-frame animation in the Sprite Editor

See the Introduction to Sprite Animations Learn tutorial.

Cutout

In cutout animation, multiple Sprites make up the body of a character, and each piece moves to give the visual effect of

By clicking "Accept Cookies", you agree to the storing of cookies on your device
to enhance site navigation, analyze site usage, and assist in our marketing efforts.

With skeletal animation, you map a Sprite or a group of Sprites onto an animation skeleton. You can create and define animation bones for characters and objects, that define how they should bend and move. This approach allows the bones to bend and deform the Sprites, for a more natural movement style. To use skeletal animation, you need to use the 2D Animation package (installed by default).

For a 2D Animation workflow, including a guide to working with the Bone Editor, see the 2D Animation documentation.

A character with bones in the Bone Editor
A character with bones in the Bone Editor

Graphics

This section describes the 2D URP graphic features available when using Universal **Render Pipeline**A series of operations that take the contents of a Scene, and displays them on a screen. Unity lets you choose from pre-built render pipelines, or write your own. More info
See in Glossary (URP).

Lighting

Because you're using URP with the 2D Renderer, you can use the Light 2D component to apply optimized 2D lighting to Sprites. For details, see Introduction to Lights 2D.

These two images show the same scene; in the image on the left, 2D Lights are disabled, and in the image on the right, 2D lights are enabled. With 2D Lights, you can use the same Sprites to create different weather conditions or moods.
These two images show the same scene; in the image on the left, 2D Lights are disabled, and in the image on the right, 2D lights are enabled. With 2D Lights, you can use the same Sprites to create different weather conditions or moods.

See the Lighting in URP Learn tutorial.

Shadows

To define the shape and properties that a Light uses to determine the shadows it casts, use the Shadow Caster 2D component.

A shadow intensity of 0.5 in the Shadow Caster 2D component
A shadow intensity of 0.5 in the Shadow Caster 2D component

Enhanced look and feel

Particle systems and **post-processing**A process that improves product visuals by applying filters and effects before the image appears on screen. You can use post-processing effects to simulate physical camera and film properties, for example Bloom and Depth of Field. More info post processing, postprocessing, postprocess
See in Glossary are optional tools that you can use to add polish to your game.

Particle systems

You can use **particle**A small, simple image or mesh that is emitted by a particle system. A particle system can display and move particles in great numbers to represent a fluid or amorphous entity. The effect of all the particles together creates the impression of the complete entity, such as smoke. More info
See in Glossary systems to create dynamic objects like fire, smoke or liquids, as an alternative to using a Sprite. Sprites are more suited to physical objects. See Particle systemsA component that simulates fluid entities such as liquids, clouds

By clicking "Accept Cookies", you agree to the storing of cookies on your device to enhance site navigation, analyze site usage, and assist in our marketing efforts.

You can use post-processing effects and full-screen effects to significantly improve the appearance of your game. For example, you can use these effects to simulate physical camera or film properties, or to create stylized visuals.

URP has its own post-processing implementation. See Post-processing in the Universal Render Pipeline.

The Lost Crypt demo uses the bloom and vignette post-processing effects
The Lost Crypt demo uses the bloom and vignette post-processing effects

Physics 2D

The Physics 2D settings define limits on the accuracy of the physical simulation in your 2D game. See 2D Physics.

This video provides an overview of 2D physics features in Unity 2020.1.

To learn how to use Unity's 2D **physics engine**A system that simulates aspects of physical systems so that objects can accelerate correctly and be affected by collisions, gravity and other forces. More info
See in Glossary, refer to Applying 2D Colliders for Physics Interactions.

The following 2D physics tools are useful for 2D games.

- Rigidbody 2D
- Collider 2D
- Triggers
- 2D Joints
- 2D Effectors

Rigidbody 2D

A **Rigidbody**A component that allows a GameObject to be affected by simulated gravity and other forces. More info
See in Glossary 2D component places a GameObject under the control of the physics engine. See Rigidbody 2D.

The Rigidbody 2D component
The Rigidbody 2D component

Collider 2D

Collider 2D components define the shape of a 2D GameObject for the purposes of physical collisions. You can also use **Collider**An invisible shape that is used to handle physical collisions for an object. A collider doesn't need to be exactly the same shape as the object's mesh - a rough approximation is often more efficient and indistinguishable in gameplay. More info
See in Glossary 2D components for input detection. For example, in mobile games you can use them to make Sprites selectable.

The Collider 2D types that you can use with Rigidbody 2D are:

- Circle Collider 2D
- Box Collider 2D
- Polygon Collider 2D
- Edge Collider 2D
- Capsule Collider 2D
- Composite Collider 2D

The Circle Collider 2D component
The Circle Collider 2D component

By clicking "Accept Cookies", you agree to the storing of cookies on your device
to enhance site navigation, analyze site usage, and assist in our marketing efforts.

The Circle Collider 2D component with **Is Trigger** selected

2D Joints

Joints attach GameObjects together. You can only attach 2D **joints**A physics component allowing a dynamic connection between Rigidbody components, usually allowing some degree of movement such as a hinge. More info
See in Glossary to GameObjects that have a Rigidbody 2D component attached, or to a fixed position in world space. See 2D Joints.

2D Effectors

Use Effector 2D componentsA functional part of a GameObject. A GameObject can contain any number of components. Unity has many built-in components, and you can create your own by writing scripts that inherit from MonoBehaviour. More info
See in Glossary with Collider 2D components to direct the forces of physics in your scene when GameObject Colliders come into contact with each other. See 2D Effectors.

Audio

You can add background music and sound effects to your game in Unity; see Audio Overview. Use third-party software to create your audio and import it into Unity with the recommended settings.

User interface

If you want to add a menu or help to your game, you need to set up a user interface. To set up a user interface, use Unity UI.

Profiling, optimizing and testing a build

Profiling

Profiling allows you to see how resource-intensive the different parts of your game are. You should always profile your game on its target release platform; see Profiling your application.

Testing

Test your game and your code with the Unity Test Framework; see Unity Test Framework.

The Test Runner window
The Test Runner window

Publishing

When you've finished your game, you're ready to publish it. See Publishing Builds.

Set up a project for 2D games with the Universal 2D template
2D game perspectives reference
Copyright ©2005-2026 Unity Technologies. All rights reserved. Built from job ID 65521287. Built on: 2026-03-26.
TutorialsCommunity AnswersKnowledge BaseForumsAsset StoreTerms of useLegalPrivacy PolicyCookiesDo Not Sell or Share My Personal Information

By clicking "Accept Cookies", you agree to the storing of cookies on your device to enhance site navigation, analyze site usage, and assist in our marketing efforts.